포팅 메뉴얼

☑️ 1. 개발 환경

✅ 1.1 FrontEnd

✔️ 1.1.1 주요 프레임워크 및 라이브러리

- **React** : `^19.1.0`
- **React DOM** : `^19.1.0`
- **React Router** : `^7.7.1`

✔️ 1.1.2 상태 관리

- **TanStack Query (React Query)** : `^5.83.0`
- **Zustand** : `^5.0.6`

✔️ 1.1.3 스타일링

- **Tailwind CSS** : `^4.1.11`
- **Headless UI** : `^2.2.6`
- **lucide-react** : `^0.532.0`
- **react-loading-skeleton** : `^3.5.0`

✔️ 1.1.4 데이터 통신 및 실시간

- **Axios** : `^1.11.0`
- **StompJS** : `^7.1.1`
- **Socket.IO** : `^4.8.1`

✔️ 1.1.5 데이터 시각화

- **Nivo** : `^0.99.0`

✔️ 1.1.6 기타 유틸리티

- **React Markdown** : `^10.1.0`
- **React Toastify** : `^11.0.5`

✔️ 1.1.7 개발 도구 및 환경

- **Vite** : `^7.0.4`
- **TypeScript** : `~5.8.3`
- **ESLint** : `^9.30.1`
- **Prettier** : `^3.6.2`
- **Husky** : `^9.1.7`
- **lint-staged** : `^16.1.2`

✅ 1.2 BackEnd

✔️ 1.2.1 개발 환경

- **IDE** : IntelliJ IDEA
- **WAS** : Embedded Tomcat **10.1.42**

✔️ 1.2.2 언어 및 핵심 프레임워크

- **Java** : **OpenJDK 21**
- **Spring Boot** : `3.5.3`
  - Spring Framework : `6.2.8`
  - Spring Security : `6.5.1`
  - Spring WebSocket : `6.2.8`

- Validation (Hibernate Validator) : `8.0.2.Final`

✔️ 1.2.3 데이터베이스 및 데이터 처리

- **Database : MySQL**
  - MySQL Connector/J : `9.2.0`
- **Spring Data JPA** : `3.5.1`
  - Hibernate ORM : `6.6.18.Final`
- **Spring Data MongoDB** : `4.5.1`
- **Spring Data Redis** : `3.5.1`
  - Lettuce (Redis Client) : `6.6.0.RELEASE`

✔️ 1.2.4 API 문서화

- **SpringDoc OpenAPI** : `2.2.0`
- **Swagger UI** : `5.2.0`

✔️ 1.2.5 테스트 및 도구

- **Gradle** : `8.14.3`
- **JUnit 5** : `5.12.2`
- **Lombok** : `1.18.38`

✅ 1.3 AI

✔️ 1.3.1 언어 및 프레임워크

- **Python** : `3.11`
- **FastAPI** : `0.116.1`
- **Uvicorn** : `0.35.0`
- **ydantic** : `2.11.7`

✔️ 1.3.2 주요 라이브러리

- **LangChain** : `0.3.27`
- **OpenAI** : `1.99.6`
- **mysql-connector-python** : `9.4.0`
- **python-dotenv** : `1.1.1`

☑️ 2. 환경 변수 설정

✅ 2.1 FrontEnd

```
VITE_APP_BASE_URL=${VITE_APP_BASE_URL}
```

✅ 2.2 BackEnd

```
spring:
  config:
    activate:
      on-profile: prod

  datasource:
    url: jdbc:mysql://${MYSQL_CONTAINER_NAME}:3306/${MYSQL_DATAB
ASE}?useSSL=false&allowPublicKeyRetrieval=true
    username: ${MYSQL_USER}
    password: ${MYSQL_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver

  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQLDialect
```

```
        format_sql: false
        show_sql: false

  data:
    redis:
      host: redis
      port: 6379
      password: ${REDIS_PASSWORD}
    mongodb:
      uri: mongodb://${MONGO_DB_USER}:${MONGO_DB_PASSWORD}@mo
ngodb:27017/${MONGO_DB_NAME}?authSource=admin

logging:
  level:
    com.sonfind.chelsea: INFO
    org.hibernate.SQL: WARN
    org.hibernate.type.descriptor.sql: WARN

upload:
  dir: /app/uploads/


custom:
  service:
    url: "https://i13a704.p.ssafy.io"
```

```
SPRING_PROFILES_ACTIVE=${SPRING_PROFILES_ACTIVE}

PROJECT_NAME=${PROJECT_NAME}
TEAM_NAME=${TEAM_NAME}

# MySQL 설정
MYSQL_VERSION=${MYSQL_VERSION}
MYSQL_CONTAINER_NAME=${TEAM_NAME}-mysql
MYSQL_HOST_PORT=3306
MYSQL_DATABASE=${PROJECT_NAME}_db
MYSQL_USER=${PROJECT_NAME}_user
MYSQL_PASSWORD=${MYSQL_PASSWORD}
```

```
MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}

# Redis 설정
REDIS_VERSION=${REDIS_VERSION}
REDIS_CONTAINER_NAME=${TEAM_NAME}-redis
REDIS_HOST_PORT=${REDIST_HOST_PORT}

# MongoDB 설정
MONGO_VERSION=${MONGO_VERSION}
MONGO_CONTAINER_NAME=${TEAM_NAME}-mongodb
MONGO_HOST_PORT=${MONGO_HOST_PORT}
MONGO_DB_NAME=${MONGO_DB_NAME}
MONGO_DB_USER=${MONGO_DB_USER}
MONGO_DB_PASSWORD=${MONGO_DB_PASSWORD}
```

✅ 2.3 AI

```
OPENAI_API_KEY=${OPEN_API_KEY}

MYSQL_VERSION=${MYSQL_VERSION}
MYSQL_CONTAINER_NAME=${TEAM_NAME}-mysql
MYSQL_HOST_PORT=3306
MYSQL_DATABASE=${PROJECT_NAME}_db
MYSQL_USER=${PROJECT_NAME}_user
MYSQL_PASSWORD=${MYSQL_PASSWORD}
MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
```

☑️ 3. 배포 환경 설정

✅ 3.1 초기 세팅

1. 타임존 변경

```
$ sudo timedatectl set-timezone Asia/Seoul # 서울로 설정
```

```
$ timedatectl # 확인
```

2. Docker 설치

```
$ sudo apt update
$ sudo apt install -y ca-certificates curl gnupg lsb-release

# Docker GPG 키 추가
$ sudo install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
$ sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Docker 저장소 추가
$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/
docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |
\
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 설치 및 서비스 실행
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-build
x-plugin docker-compose-plugin

# 도커 권한 설정 (재로그인 필요)
$ sudo usermod -aG docker $USER
```

3. 루트 권한 없이 Docker 실행

```
$ sudo groupadd docker
$ sudo usermod -aG docker $USER

// 재로그인 후 실행
$ newgrp docker
```

4. Docker 서비스 시작 및 부팅 시 자동 시작 설정

```
$ sudo systemctl start docker # Docker 서비스 즉시 시작
$ sudo systemctl enable docker # 서버 부팅 시 Docker가 자동으로 실행되어 컨테이너가 자동으로 구동되도록 설정
$ sudo systemctl status docker # 도커 상태 확인
```

5. Docker compose 설치

```
$ sudo curl -L "https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
$ sudo chmod +x /usr/local/bin/docker-compose
$ docker-compose --version # 설치 확인
```

6. Jenkins 내부 node js 설치 Dockerfile 생성

```
FROM jenkins/jenkins:lts

USER root

RUN apt-get update && apt-get install -y curl

RUN curl -fsSL https://download.docker.com/linux/static/stable/x86_64/docker-26.1.4.tgz | tar --strip-components=1 -xz -C /usr/local/bin docker/docker

RUN mkdir -p /usr/local/lib/docker/cli-plugins
RUN curl -SL https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-linux-x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose
RUN chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

RUN curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
RUN apt-get install -y nodejs

USER jenkins
```

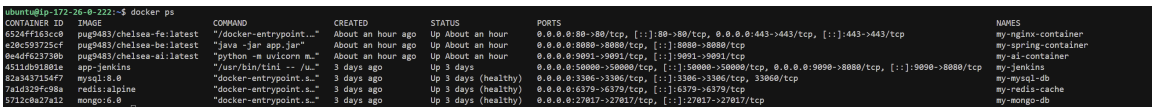
7. Nginx certbot 설치

```
$ sudo apt-get install letsencrypt
$ sudo apt-get install certbot python3-certbot-nginx -y

$ sudo certbot certonly --standalone -d i13a704.p.ssafy.io

$ sudo service nginx restart
$ sudo systemctl reload nginx
```

8. 방화벽

Port 번호	내용
22	SSH
80	HTTP
443	HTTPS
8080/tcp	Spring Boot
8090/tcp	Jenkins
9090	Fast API
3306	MySQL
27017	MongoDB

9. 실행 환경



✅ 3.2 Docker 컨테이너 생성

✔️ 3.2.1 Jenkins

```
jenkins:
  build: .
  container_name: "my-jenkins"
  restart: always
  ports:
    - "9090:8080"
    - "50000:50000"
  volumes:
```

```
    - /home/ubuntu/jenkins-data:/var/jenkins_home
    - /var/run/docker.sock:/var/run/docker.sock
    - /home/ubuntu/app:/app
  networks:
    - my_network
  environment:
    - TZ=Asia/Seoul
```

✔️ 3.2.2 Mysql

```
mysql:
  image: "mysql:8.0"
  container_name: "my-mysql-db"
  restart: always
  environment:
    - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
    - MYSQL_DATABASE=${MYSQL_DATABASE}
    - MYSQL_USER=${MYSQL_USER}
    - MYSQL_PASSWORD=${MYSQL_PASSWORD}
  volumes:
    - mysql_data:/var/lib/mysql
    - /home/ubuntu/mysql/my.cnf:/etc/mysql/conf.d/my.cnf
  ports:
    - "3306:3306"
  networks:
    - my_network
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    timeout: 20s
    retries: 10
```

✔️ 3.2.3 MongoDB

```
mongo:
  image: "mongo:6.0"
  container_name: "my-mongo-db"
```

```
  restart: always
  environment:
    - MONGO_INITDB_ROOT_USERNAME=${MONGO_INITDB_ROOT_USERN
AME}
    - MONGO_INITDB_ROOT_PASSWORD=${MONGO_INITDB_ROOT_PASSW
ORD}
    - MONGO_INITDB_DATABASE=${MONGO_INITDB_DATABASE}
  volumes:
    - mongodb_data:/data/db
  networks:
    - my_network
  ports:
    - "27017:27017"
  healthcheck:
    test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test -
-quiet
    interval: 10s
    timeout: 10s
    retries: 5
    start_period: 40s
```

✔️ 3.2.4 Redis

```
redis:
  image: "redis:alpine"
  container_name: "my-redis-cache"
  restart: always
  command: redis-server --requirepass ${REDIS_PASSWORD}
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  networks:
    - my_network
  healthcheck:
    test: ["CMD", "redis-cli", "--raw", "ping"]
```

```
    interval: 10s
    timeout: 5s
    retries: 5
```

✔️ 3.2.5 Nginx

```
nginx-proxy:
  image: "pug9483/chelsea-fe:latest"
  container_name: "my-nginx-container"
  restart: always
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - /etc/letsencrypt:/etc/letsencrypt:ro
    - /var/www/certbot:/var/www/certbot
  depends_on:
    - spring-app
  networks:
    - my_network
```

✔️ 3.2.6 Spring

```
spring-app:
  image: "pug9483/chelsea-be:latest"
  container_name: "my-spring-container"
  restart: always
  ports:
    - "8080:8080"
  environment:
    - SPRING_PROFILES_ACTIVE=prod
    - SPRING_DATASOURCE_URL=jdbc:mysql://${MYSQL_CONTAINER_NAM
E}:3306/${MYSQL_DATABASE}?useSSL=false&allowPublicKeyRetrieval=tru
e
    - SPRING_DATASOURCE_USERNAME=${MYSQL_USER}
```

```
    - SPRING_DATASOURCE_PASSWORD=${MYSQL_PASSWORD}
    - SPRING_REDIS_HOST=my-redis-cache
    - SPRING_REDIS_PORT=6379
    - SPRING_REDIS_PASSWORD=${REDIS_PASSWORD}
    - SPRING_DATA_MONGODB_URI=mongodb://${MONGO_INITDB_ROOT_
USERNAME}:${MONGO_INITDB_ROOT_PASSWORD}@my-mongo-db:2701
7/${MONGO_INITDB_DATABASE}?authSource=admin
  volumes:
    - /home/ubuntu/app/storage:/app/uploads
  networks:
    - my_network
```

✔️ 3.2.7 AI

```
ai-app:
  image: "pug9483/chelsea-ai:latest"
  container_name: "my-ai-container"
  restart: always
  ports:
    - "9091:9091"
  env_file:
    - ./.env
  networks:
    - my_network
```

실행 방법

```
$ cd /home/ubuntu/app
$ docker compose -f docker-compose-infra.yml up -d
$ docker compose -f docker-compose-app.yml up -d
```

☑️ 4. 배포 이미지

✅ 4.1 FrontEnd Dockerfile

```
FROM node:22-alpine as builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

ARG VITE_APP_BASE_URL

ENV VITE_APP_BASE_URL=$VITE_APP_BASE_URL

RUN npx vite build

FROM nginx:stable-alpine

RUN rm -rf /etc/nginx/conf.d/default.conf

COPY --from=builder /app/dist /usr/share/nginx/html

COPY --from=builder /app/nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

```
limit_req_zone $binary_remote_addr zone=sse:40m rate=100r/s;

server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name i13a704.p.ssafy.io;
```

```
    root /usr/share/nginx/html;
    index index.html index.htm;

    ssl_certificate /etc/letsencrypt/live/i13a704.p.ssafy.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/i13a704.p.ssafy.io/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    client_max_body_size 55M;

    location /api/ {
        proxy_pass http://spring-app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/v1/ws-stomp {
        proxy_pass http://spring-app:8080;

        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/v1/notifications/subscribe {
        limit_req zone=sse burst=200 nodelay;

        proxy_pass http://spring-app:8080/api/v1/notifications/subscribe;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host $server_name;

        # SSE 핵심 설정
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        chunked_transfer_encoding off;
        proxy_buffering off;
        proxy_cache off;

        # SSE 장시간 연결 유지
        proxy_read_timeout 24h;
        proxy_send_timeout 24h;
        proxy_connect_timeout 30s;

        # CORS 헤더 (SSE용)
        add_header Access-Control-Allow-Origin "https://i13a704.p.ssafy.io" a
lways;
        add_header Access-Control-Allow-Methods "GET, OPTIONS" always;
        add_header Access-Control-Allow-Headers "Authorization, Content-T
ype, Accept, Cache-Control" always;
        add_header Access-Control-Allow-Credentials true always;

        # OPTIONS preflight 처리
        if ($request_method = 'OPTIONS') {
            add_header Access-Control-Allow-Origin "https://i13a704.p.ssafy.i
o" always;
            add_header Access-Control-Allow-Methods "GET, OPTIONS" alway
s;
            add_header Access-Control-Allow-Headers "Authorization, Content
-Type, Accept, Cache-Control" always;
            add_header Access-Control-Allow-Credentials true always;
            add_header Content-Length 0;
            add_header Content-Type text/plain;
            return 204;
        }
    }
```

```
    location /fastapi/ {
        proxy_pass http://ai-app:9091;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}

server {
    listen 80;
    listen [::]:80;
    server_name i13a704.p.ssafy.io;

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}
```

✅ 4.2 BackEnd Dockerfile

```
FROM openjdk:21-slim AS builder

WORKDIR /app

COPY build/libs/chelsea-0.0.1-SNAPSHOT.jar app.jar

FROM openjdk:21-slim
```

```
WORKDIR /app

ENV TZ=Asia/Seoul

COPY --from=builder /app/app.jar .

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

✅ 4.3 AI Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 9091

CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "9
091"]
```

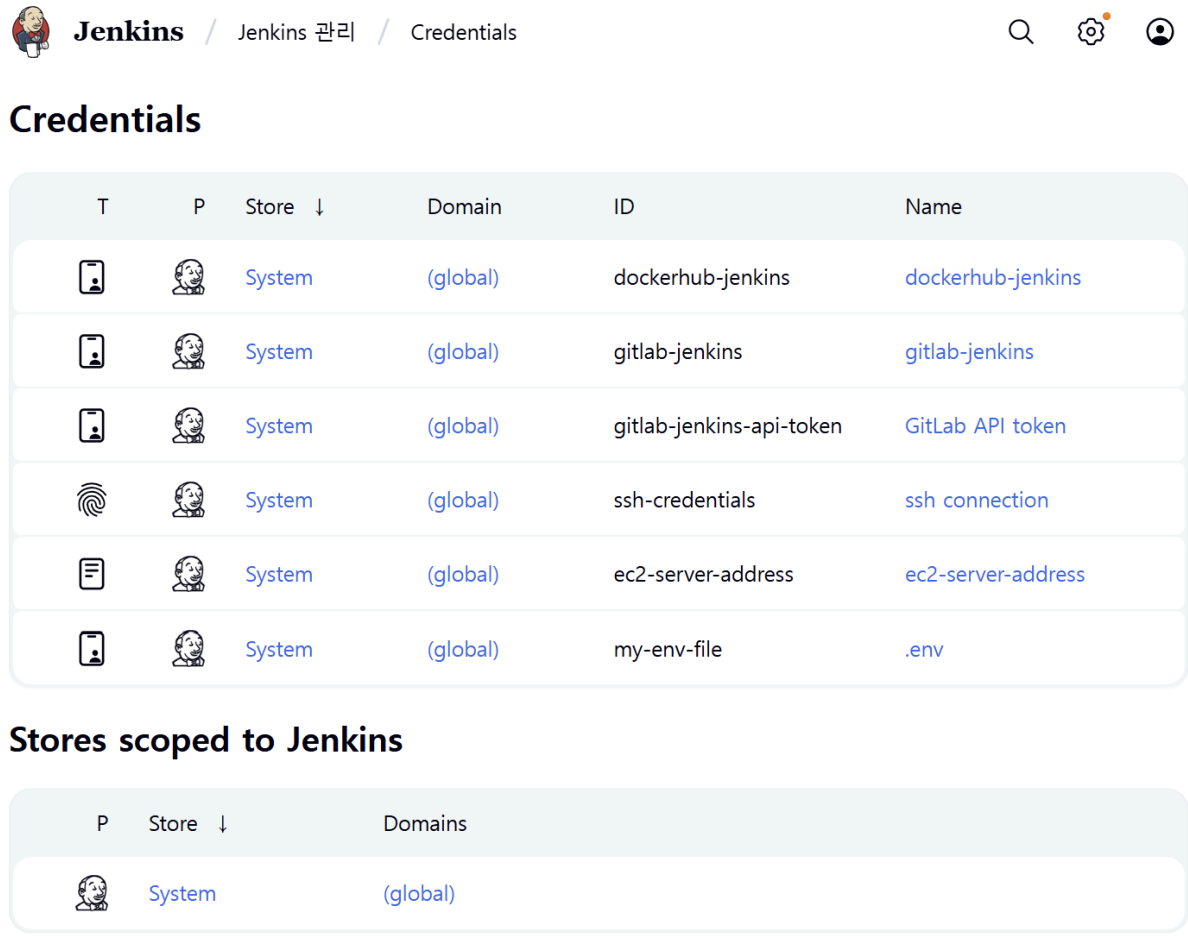
☑️ 5. Jenkins 설정

✅ 5.1 Jenkins Plugin

- SSH Agent Plugin
- SSH Server
- Pipeline Graph View
- Pipeline

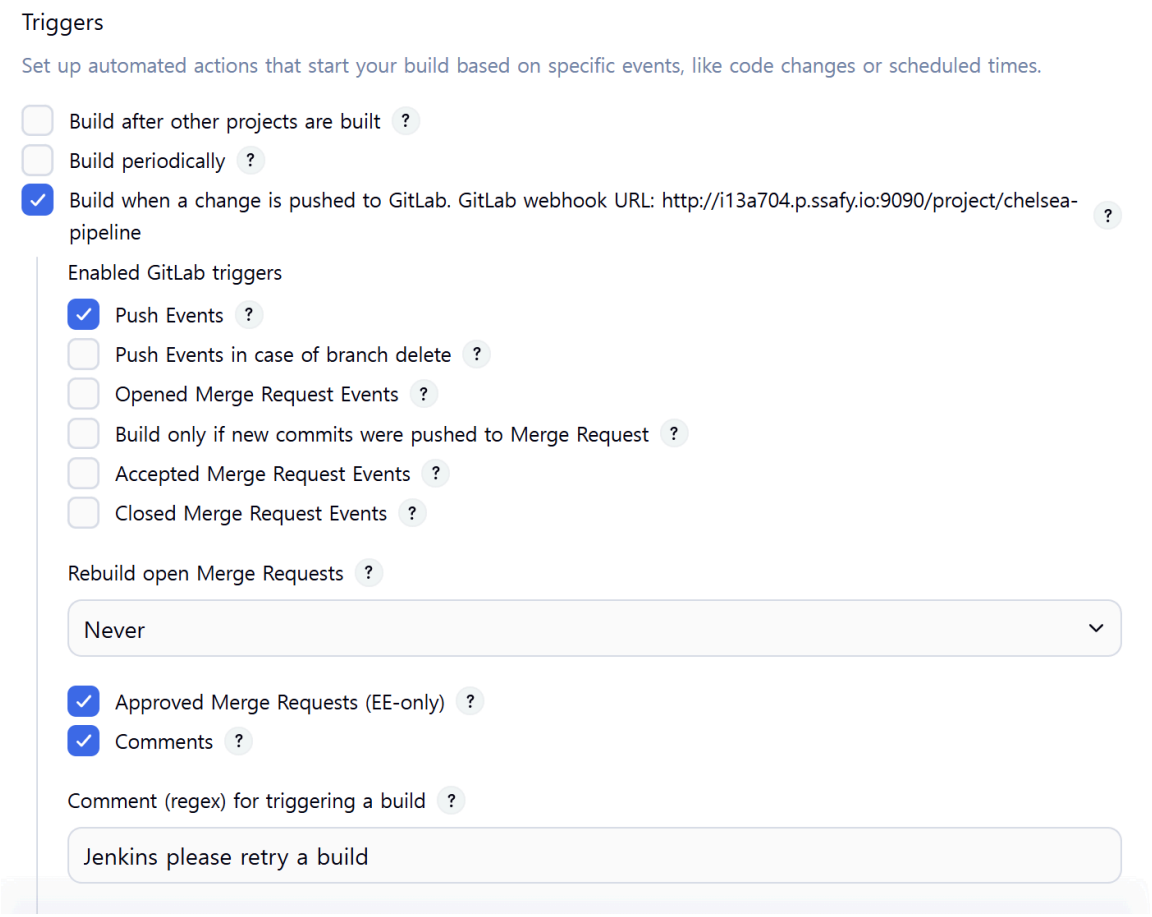
- NodeJS Plugin
- Oracle JAVA SE Development Kit installer Plugin
- Gradle Plugin

✅ 5.2 암호키 설정

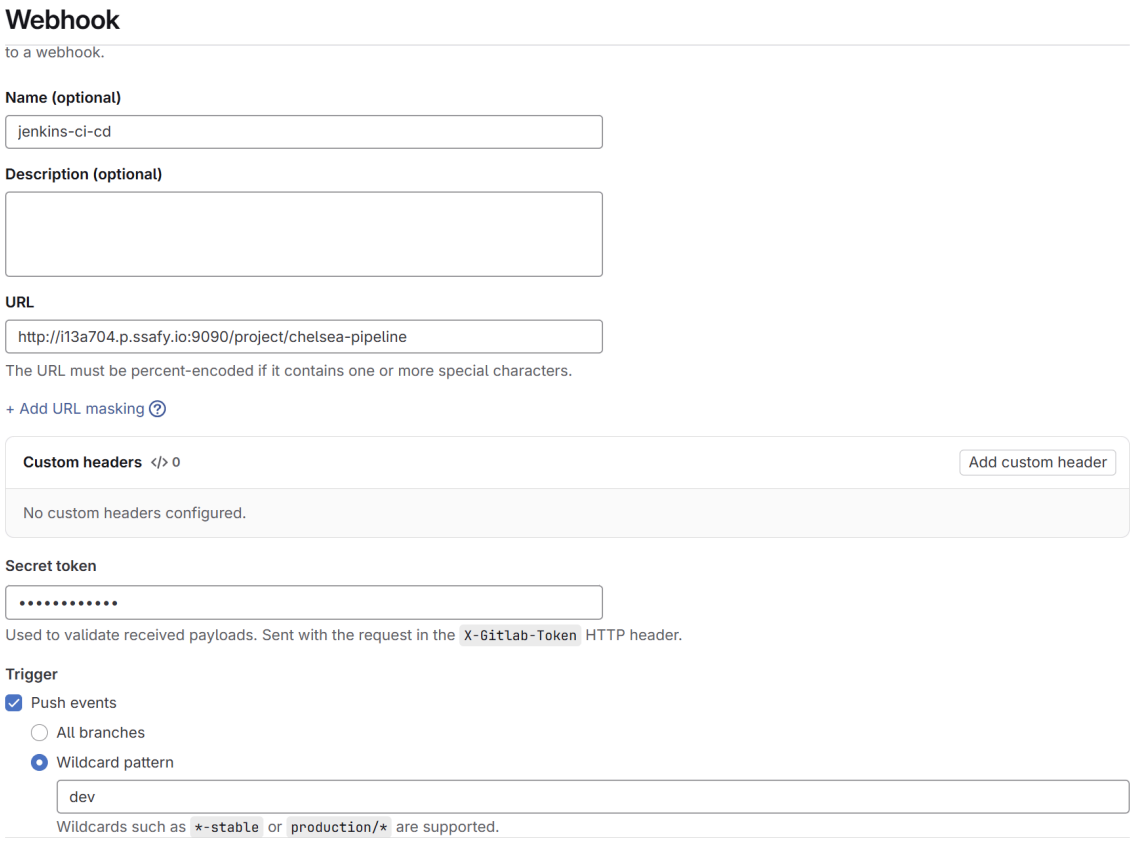


✅ 5.2 Jenkins Trigger 설정

1. Jenkins Trigger 설정



2. gitlab webhook 설정



✅ 5.3 Jenkins Pipeline

```
pipeline {
    agent any

    environment {
        PROJECT_DIR = '.'
        FRONTEND_DIR = "${PROJECT_DIR}/FE"
        BACKEND_DIR = "${PROJECT_DIR}/BE/chelsea"
        AI_DIR = "${PROJECT_DIR}/AI"

        DOCKER_FRONTEND_IMAGE = 'pug9483/chelsea-fe'
        DOCKER_BACKEND_IMAGE = 'pug9483/chelsea-be'
        DOCKER_AI_IMAGE = 'pug9483/chelsea-ai'
        DOCKER_HUB_CREDENTIAL_ID = 'dockerhub-jenkins'
        IMAGE_TAG = 'latest'

        VITE_APP_BASE_URL = 'https://i13a704.p.ssafy.io/api/v1'
```

```
    }

    stages {
        stage('Checkout') {
            steps {
                cleanWs()
                git url: 'https://lab.ssafy.com/s13-webmobile1-sub1/S13P11A704.gi
t',
                    branch: 'dev',
                    credentialsId: 'gitlab-jenkins'
            }
        }

        stage('Build Backend') {
            steps {
                echo '1. Backend 애플리케이션을 Gradle로 빌드합니다.'
                dir(BACKEND_DIR) {
                    sh 'chmod +x ./gradlew'
                    sh './gradlew clean build -x test -x checkstyleMain -x checkstyl
eTest'
                }
            }
        }

        stage('Build & Push Docker Images') {
            steps {
                echo '2. Frontend와 Backend 도커 이미지를 빌드하고 푸시합니다.'
                script {
                    withCredentials([usernamePassword(credentialsId: DOCKER_H
UB_CREDENTIAL_ID, usernameVariable: 'DOCKER_USERNAME', passwordV
ariable: 'DOCKER_PASSWORD')]) {
                        sh 'echo $DOCKER_PASSWORD | docker login -u $DOCKER_
USERNAME --password-stdin'
                    }

                    dir(FRONTEND_DIR) {
                        sh "docker build --build-arg VITE_APP_BASE_URL=${VITE_A
PP_BASE_URL} -t ${DOCKER_FRONTEND_IMAGE}:${IMAGE_TAG} ."
```

```
                sh "docker push ${DOCKER_FRONTEND_IMAGE}:${IMAGE_T
AG}"
            }

            dir(BACKEND_DIR) {
                sh 'cp `find ./build/libs -name "*.jar" | grep -v "plain.jar"` ./ap
p.jar'
                sh "docker build -t ${DOCKER_BACKEND_IMAGE}:${IMAGE_
TAG} ."
                sh "docker push ${DOCKER_BACKEND_IMAGE}:${IMAGE_TA
G}"
            }

            dir(AI_DIR) {
                sh "docker build -t ${DOCKER_AI_IMAGE}:${IMAGE_TAG} ."
                sh "docker push ${DOCKER_AI_IMAGE}:${IMAGE_TAG}"
            }
        }
    }
}

    stage('Deploy on EC2') {
        steps {
            echo 'SSH Agent를 사용하여 EC2에 재배포합니다.'
            sshagent(credentials: ['ssh-credentials']) {
                sh """
                    ssh -o StrictHostKeyChecking=no ubuntu@172.26.0.222 '''
                        # docker-compose-app.yml 파일이 있는 경로로 이동합니다.
                        cd /home/ubuntu/app

                        echo "docker-compose-app.yml을 사용하여 배포를 시작합니
다."

                        docker compose -f docker-compose-app.yml pull

                        docker compose -f docker-compose-app.yml up -d --forc
e-recreate --no-deps
```

```
                    echo "배포가 완료되었습니다."
                '''
            """
        }
    }
  }

  post {
    success {
      echo '✅ 파이프라인이 성공적으로 종료되었습니다. 작업 공간을 정리합니다.'
      cleanWs()
    }
    failure {
      echo '❌ 파이프라인이 실패했습니다. 디버깅을 위해 작업 공간을 보존합니다.'
    }
  }
}
```

☑️ 6. EC2 파일 구조

```
$ pwd
/home/ubuntu

$ tree
.
├── Dockerfile
├── docker-compose-app.yml
├── docker-compose-infra.yml
├── .env
├── 📁 mysql
│   └── conf.d
└── 📁 storage
    ├── 📁 portfolios
```

```
├─ 📁 profiles
└─ 📁 team
```